DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
[INSTITUTION / COLLEGE NAME]

PROJECT BASED LEARNING REPORT

ON

POWER-AWARE CPU SCHEDULING SIMULATOR

Subject: Software Engineering Lab [course code, if applicable]

BACHELOR OF TECHNOLOGY
IN COMPUTER SCIENCE & ENGINEERING

SUBMITTED BY
[Student name] | [Roll number] | [Year / Batch]

GUIDED BY
[Guide / faculty name]

TABLE OF CONTENTS

1. Abstract

2. Introduction

3. Problem Statement

4. Objective

5. Methodology / System Architecture

6. Requirement Analysis

7. Tools and Technologies Used

8. Implementation

9. Results

10. Purpose of Work

11. Future Scope

12. Conclusion

13. References

This report follows the section sequence and academic presentation style of the supplied Software Engineering PBL report. Student, institution and guide details are deliberately left for completion.

ABSTRACT

The Power-Aware CPU Scheduling Simulator is an interactive educational application that compares CPU scheduling decisions using two competing measures: energy consumption and job response time. It models a single CPU, user-defined jobs, discrete frequency and power states, idle energy, and context-switch overhead. Three policies are implemented: First Come, First Served (FCFS), Round Robin (RR), and a power-aware Round Robin variant that chooses CPU frequency at each dispatch using a configurable weighted score.

The application validates workloads, runs deterministic event-based simulations, and presents comparable policy summaries, per-job times, a replayable execution timeline, energy breakdowns, a weight-sweep chart, and downloadable JSON. A Node.js API powers local and serverless deployments; the GitHub Pages edition runs the same calculation module directly in the browser because static hosting cannot execute the backend.

For the included three-job example, FCFS uses 48.2 mJ with 4.87 ms mean response, RR uses 48.8 mJ with 1.27 ms mean response, and the power-aware policy at weight 0.5 uses 49.4 mJ with 1.27 ms mean response. These values demonstrate that a lower frequency does not necessarily reduce total energy. They are simulation outputs, not physical measurements.

Keywords: CPU scheduling, FCFS, Round Robin, dynamic frequency selection, energy modelling, response time, discrete-event simulation, web application.

INTRODUCTION

CPU scheduling determines when ready jobs receive processor time. A scheduler favouring quick first response may incur more context switches or higher active power; a scheduler favouring low-power execution may delay completion. Examining only execution time or only active power therefore gives an incomplete picture.

This project provides a reproducible environment in which the same workload and machine assumptions are supplied to three scheduling policies. Users can edit job arrivals and work, frequency/power states, the scheduling quantum, and the energy weight. The interface shows both aggregate comparisons and the underlying timeline so that an observed trade-off can be explained rather than treated as a black-box result.

- Scope: a single CPU and CPU-bound jobs with known arrival time and work.
- Units: simulated milliseconds (ms), watts (W), and millijoules (mJ).
- Boundaries: no real hardware sensing, multi-core execution, I/O blocking or optimality guarantee.

PROBLEM STATEMENT

Introductory CPU scheduling comparisons often emphasize waiting time or throughput without accounting for processor energy. Conversely, choosing a low-power CPU state can lengthen execution and increase accumulated energy or response delay. Students need a way to inspect how a given machine model and workload produce these effects.

Problem statement

Develop a web-accessible, deterministic scheduling simulator that compares FCFS, Round Robin and a tunable power-aware policy on identical inputs, calculates energy and first-start response time, and makes the results understandable through visual and tabular outputs.

- Traditional scheduling examples may not quantify active, idle and switch energy separately.
- A frequency that is cheaper per unit time need not be cheaper for completing a job.
- A single metric cannot express the complete energy-versus-responsiveness trade-off.
- Unvalidated inputs and ambiguous timing rules make comparisons hard to reproduce.

OBJECTIVE

The primary objective is to build and explain a scheduling model that makes the tension between simulated CPU energy and response time measurable.

- Implement one deterministic event engine shared by all policies.
- Model job arrivals, CPU work, dispatches, idle intervals and context switches.
- Implement FCFS, quantum-based Round Robin and a frequency-selecting power-aware scheduler.
- Compute per-job first-start response, turnaround and waiting time.
- Compute total energy and active, idle and switch components from power × duration.
- Provide a responsive web interface for editing settings, running simulations and replaying schedules.
- Support weight sweeps, policy comparison, accessible data tables and JSON export.
- Validate inputs and test timing, energy and API behaviour with automated tests.

METHODOLOGY / SYSTEM ARCHITECTURE

A shared simulation contract defines the input JSON, units, tie-breaking, event timing and metrics. The implementation separates policy decisions from the engine that advances simulated time. The comparison runner executes all policies on the same validated input and packages their timelines and metrics. The interface submits work to a Node.js endpoint where available; on GitHub Pages it invokes the same calculation module in-browser.

Layer	Responsibility
Input & validation	Jobs, arrivals, work, machine states, quantum, energy weight and sweep weights.
Policy module	FCFS and RR baseline selection; power-aware frequency scoring.
Simulation engine	Ready queue, arrivals, run/idle/switch events, timing and energy accounting.
Comparison & output	Three policy results, tested-point frontier and serializable JSON.
Web UI / API	Editor, summaries, replay, charts, import/export; Node and Vercel endpoints.

Simulation flow

Load or edit workload → validate fields → select policy → process arrivals and dispatches → record run/idle/switch intervals → compute per-job and aggregate metrics → render comparison and export JSON.

Feasibility

Technical: JavaScript ES modules and a Node.js 20+ runtime require no third-party runtime dependencies. Economic: source code and GitHub Pages can be hosted without dedicated hardware. Operational: the interface provides a bundled workload and editable fields for educational experiments.

REQUIREMENT ANALYSIS

Users and activities

User	Main activities
Student	Modify a workload; compare policies; study the timeline and response-time calculation.
Educator	Demonstrate scheduling rules and discuss energy/latency trade-offs.
Developer / researcher	Import JSON workloads, sweep weights, download full-precision results.

Functional requirements

- Accept 1–200 jobs with unique IDs, nonnegative arrivals and positive work.
- Accept sorted, positive CPU frequency states with nondecreasing active power.
- Run FCFS, RR and power-aware scheduling with deterministic queue ordering.
- Report policy metrics, per-job values, the full timeline and a weight sweep.
- Permit JSON import and results export; provide visible errors for invalid fields.

Non-functional requirements

Repeatability: identical inputs produce identical event order and numerical output. Usability: responsive controls, labelled charts and equivalent data tables. Maintainability: a pure simulation engine shared by the website and APIs. Accuracy within the model: all intervals have positive duration and total energy matches the sum of interval energy. Security: no user accounts or persistent personal data are needed.

TOOLS AND TECHNOLOGIES USED

Purpose	Tool / technology
Simulation and policies	JavaScript ES modules (src/engine.js and src/policies.js)
Local HTTP and calculation API	Node.js HTTP server (server.js)
Serverless calculation API	Vercel Node.js Function (api/simulate.js)
Website	HTML5, CSS, JavaScript and SVG
Static hosting	GitHub Pages; browser-side calculation
Automated tests	Node.js built-in test runner (npm test)
Data interchange	JSON workload, settings, timeline and results

No database or physical CPU sensor is used. The model operates entirely on the supplied machine profile and jobs.

IMPLEMENTATION

Scheduling rules

FCFS runs the oldest ready job to completion. RR uses a FIFO ready queue and a wall-clock quantum; arrivals at a slice boundary are queued before the unfinished job. The power-aware policy uses the same RR rotation but chooses an allowed CPU frequency at each dispatch. Baseline policies use the frequency nearest 1.0, breaking an exact tie toward the lower frequency. A switch interval is charged when consecutive run segments belong to different jobs; idle and switch intervals use idle power.

Power and frequency model

At frequency f, a job with work w takes w/f milliseconds without interruption. For an interval, energy in millijoules equals power in watts multiplied by interval duration in milliseconds. Response time is first CPU start minus arrival; turnaround is completion minus arrival; waiting is turnaround minus actual running time.

Power-aware local score

For each candidate frequency, the scheduler estimates active energy E = active power × min(quantum, remaining work / frequency). It also estimates extra first-response delay D for other jobs not yet started that are waiting or arrive during that slice. A weighted normalized score is minimized: weight × E/Escale + (1 - weight) × D/Dscale. The scales depend on the input machine and workload, not measured outcomes. The heuristic selects locally and does not promise a globally optimal schedule.

User-facing workflow

The application preloads an example, accepts edited or imported JSON input, runs a comparison, presents policy summaries and a schedule replay, and downloads a full-precision JSON result. On the hosted Pages edition, the computation is local to the browser; local Node/Vercel editions use the calculation API.

RESULTS

The following reproducible run uses the bundled example: A arrives at 0 ms with 8 work units; B arrives at 1 ms with 3; C arrives at 4 ms with 5. Frequencies are 0.5/1.0/1.5 with active powers 1.5/3/6 W; idle power is 0.5 W, switch duration 0.2 ms, RR quantum 2 ms and power-aware weight 0.5.

Policy	Energy (mJ)	Mean response (ms)	Max response (ms)	Finish (ms)
FCFS	48.20	4.87	7.40	16.40
Round Robin	48.80	1.27	2.60	17.60
Power-aware	49.40	1.27	2.60	31.80

FCFS finishes first and has the lowest total energy in this example, but later jobs wait longer before their first execution. RR reduces mean first-response time from 4.87 ms to 1.27 ms at a small energy cost. The power-aware run also yields 1.27 ms mean response, but finishes much later and uses 49.4 mJ. A lower selected frequency does not save total energy here, because active execution and switching continue longer.

Selected trace: FCFS

Interval (ms)	Event	Frequency	Energy (mJ)
0–8	Run A	1.0	24.0
8–8.2	Switch	—	0.1
8.2–11.2	Run B	1.0	9.0
11.2–11.4	Switch	—	0.1
11.4–16.4	Run C	1.0	15.0

Energy check: 24 + 0.1 + 9 + 0.1 + 15 = 48.2 mJ. The five configured sweep weights all produce the same power-aware result on this example; no changing trade-off is claimed for this workload.

Automated verification: 18 tests passed at report generation time, covering input validation, event ordering, energy accounting, policy decisions, comparison output, local/Vercel endpoints and GitHub Pages path handling.

PURPOSE OF WORK

The project turns CPU scheduling from an abstract formula exercise into a reproducible experiment. It helps learners compare scheduling algorithms fairly, observe the effect of dispatch and frequency decisions, and understand why minimizing one metric can worsen another.

- Enable hands-on experimentation with arrival time, work, quantum, power states and energy weight.
- Make response time distinct from turnaround time in both per-job and aggregate outputs.
- Illustrate that total energy depends on duration and switching as well as active power.
- Support explanation through event tables, playback, energy breakdowns and JSON evidence.
- Apply software-engineering practices: requirements, modular architecture, validation and tests.

FUTURE SCOPE

- Introduce multi-core processors, affinity and shared-power constraints.
- Add realistic measured hardware power profiles and frequency-transition overhead.
- Model I/O blocking, interactive bursts, deadlines and additional scheduling baselines.
- Introduce a configurable response-time constraint and broader multi-objective policy search.
- Add seeded synthetic workload generators and repeatable bulk experiments.
- Add performance limits, background computation and richer accessibility audits for large workloads.

These extensions are proposals, not features of the current single-CPU simulator.

CONCLUSION

The Power-Aware CPU Scheduling Simulator implements a deterministic, single-CPU scheduling model with an accessible website and shared calculation logic. It compares FCFS, Round Robin and a tunable power-aware variant using recorded CPU intervals, per-job response times and interval-based energy accounting. The built-in example confirms that lower mean response can cost energy and that low-frequency execution is not automatically an energy-saving choice.

The implementation combines requirement analysis, model specification, modular scheduling logic, validation, visual presentation and automated testing. It is suitable for understanding trade-offs under stated assumptions, not for measuring real processor energy or proving optimal scheduling. The hosted GitHub Pages edition provides an immediately usable experiment environment.

REFERENCES

A. Silberschatz, P. B. Galvin and G. Gagne, Operating System Concepts, Wiley (CPU scheduling concepts).

Repository: Mayank17OP/Power-Aware-CPU-Scheduling-Simulator, https://github.com/Mayank17OP/Power-Aware-CPU-Scheduling-Simulator

Project requirements and model specification: PRD.md and docs/SPEC.md in the project repository.

Web UX design and example workload: docs/UX.md and examples/default.json in the project repository.

Node.js documentation, https://nodejs.org/docs/latest/api/ (HTTP server and test runner).

GitHub Pages documentation, https://docs.github.com/en/pages (static hosting).

Vercel Functions documentation, https://vercel.com/docs/functions (serverless deployment).

Live application: https://mayank17op.github.io/Power-Aware-CPU-Scheduling-Simulator/